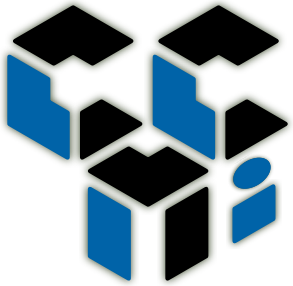
OCL: Object Constraint Language





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

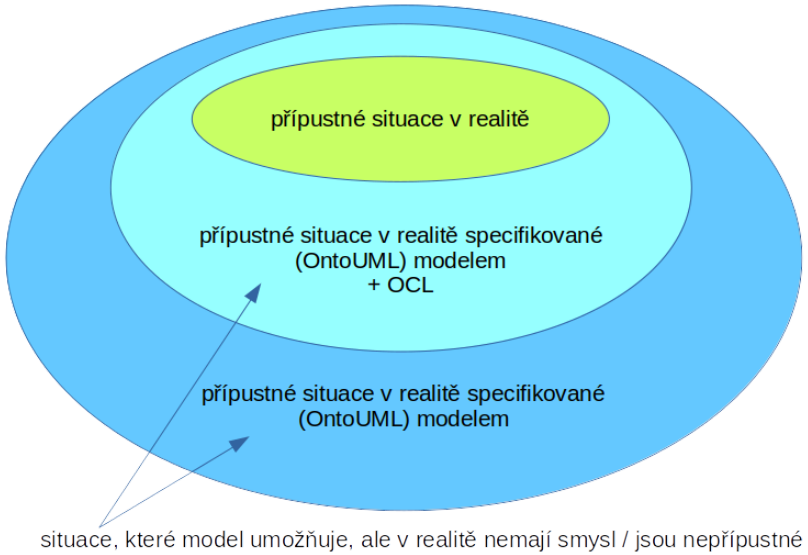
BI-KOM: Konceptuální modelování #7

doc. Ing. Robert Pergl, Ph.D. (robert.pergl@fit.cvut.cz)

Obsah přednášky

1. Motivace
2. Integritní omezení v DB
3. Jazyky pro specifikace pravidel v modelech
4. Jazyk OCL - syntaxe a příklady
5. Nástroje pro OCL

Pravdivost modelu



Integritní omezení v relačních databázích

- (flashback z BI-DBS) (R, I) je schéma relační databáze, kde:
  - $R = R_1, R_2, \ldots, R_n$ je množina relací a
  - I je množina **integritních omezení** (IO).
- Omezení na úrovni hodnot: *Známka je číslo od 1 do 5.*
- Omezení vztahů: *Jeden film se nehraje více než ve třech kinech.*
- Složitější omezení: *Řidič nesmí mít překrývající se směny.*
- Cílem je zamezit vzniku nepřípustných instancí v databázi = zachovat pravdivost

Specifikace omezení

- Existuje řada jazyků a metod pro specifikaci pravidel a validaci modelů.
- Obecně se tyto jazyky označují jako *Specification language*.
- Založené na algebraickém popisu modelu a dalších matematických aparátech.
- Některé umožňují i simulace a formální validace implementujícího software (implementace modelu).
- Příklady:
  - **Alloy** = specifikace *micro-models* založené na teorii množin a predikátové logice pro simulaci a validaci; používán např. v OntoUML nástroji OLED.
  - **Vienna Development Method (VDM)** = rozsáhlá, jedna z nejdéle používaných metod, původně od IBM.
  - **Z-Notation** = komplexní matematická notace pro popis systémů založená na teorii množin, lambda kalkulu a predikátové logice.

OCL: Object Constraint Language

- Standardní dotazovací jazyk **UML 2** sloužící pro **specifikaci omezení** v modelech, která nelze vyjádřit v modelovací notaci samotné:
  - **invariants** = to, co se nemění (musí platit vždy)
  - **pre-/post-conditions** = to, co musí platit *před* a *po* vykonání metody
  - ... a další
- Deklarativní jazyk = množina definic
- Pouze dotazovací = nemůže měnit model a jeho instance (side-effect free)

OCL: Object Constraint Language

- Základem je **tvrzení (assertion)** = predikát (má vždy hodnotu `true` nebo `false`), o kterém předpokládáme, že je vždy pravda a tuto pravdu chceme kontrolovat.
  - Známý konstrukt z programovacích jazyků: assert statement, contracts (Eiffel, Racket), Clojure(Script) Schema, ...
- Slouží pro kontrolu, že model je **well-formed** (dobře postavený).
- Lze použít pro UML modely i metamodely, viz např. [Benevides] či [Guerson] pro OntoUML
- Jelikož OntoUML je profil jazyka UML, OCL funguje pro něj stejně.
- Existuje i řada rozšíření (např. časová omezení).
- Používán v dalších OMG jazycích (např. QVT).

OCL: Historie

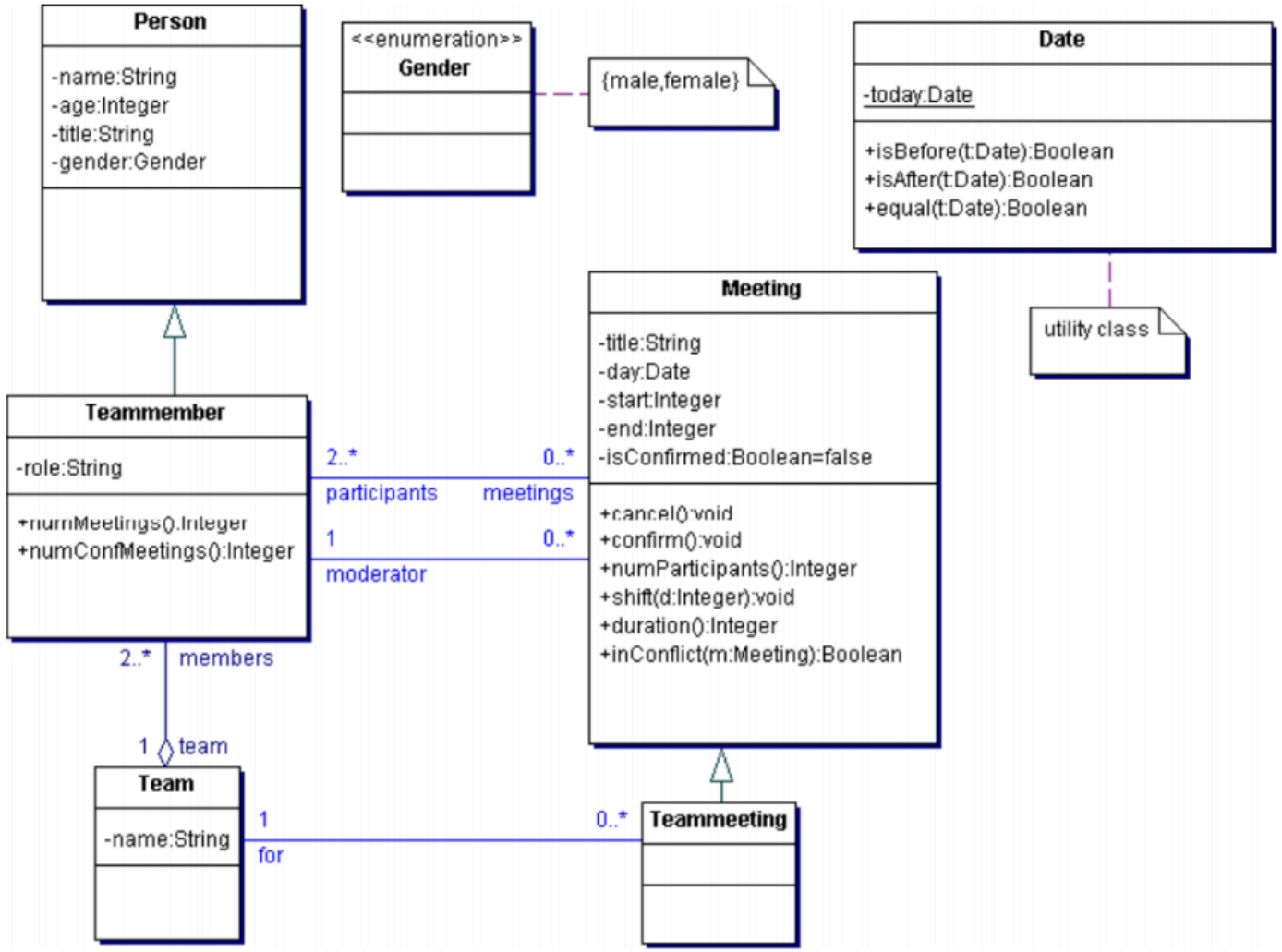
- Nástupce metody *Syntropy* pro objektově-orientovanou analýzu z 90. let.
- Vznikl v roce 1995 v IBM jako *business engineering language*.
- Od roku 1997 integrován s UML (verze 1.1).
- Původně jen jako jazyk pro definici omezení.
- Od verze 2.0 plně dotazovací jazyk.
- Aktuální verze je OCL 2.4 z roku 2014.

OCL: Omezení (Constraint)

- Pomocí *assertions* tvoříme **Omezení objektů (Object constraints)*** v modelech.
- Omezení jedné nebo více hodnot.
- Formulovány na úrovni *tříd* a aplikovány pro *instance*.
- Omezení se vždy vztahuje k určitému `context` (např. třída, metoda, atribut).

Pozn.: (*) Jelikož se nyní pohybujeme v UML, objekt je třeba chápat jako instanci OO třídy.

OCL: Model pro příklady



OCL: Invariant

- Invariant je constraint, který má být pravdivý během celého života modelu.
- Obvykle odráží *doménová pravidla (domain rules)*, též nazývaná *podniková pravidla (business rules)*.

```
context <classifier>
inv [<constraint_name>]: <Boolean_OCL_expression>
```

OCL: Invariant (příklady)

```
context Meeting
inv: self.end > self.start
```

- `self` ukazuje na objekt, ze kterého je constraint vyhodnocován, lze implicitně:

```
context Meeting
inv: end > start
```

- Constraint lze pojmenovat:

```
context Meeting
inv startEndConstraint : end > start
```

- Slouží pro identifikaci constraintu při vyhodnocování

OCL: Precondition / Postcondition

- Constraint, který specifikuje aplikovatelnost a výsledek operace bez ohledu na jeho algoritmus či implementaci.
- Přidává se k operaci ve třídě.
- **Precondition** = musí být splněn **před** vykonáním operace

```
context <classifier>::<operation>(<parameters>)
pre [<constraint_name>]: <Boolean_OCL_expression>
```

- **Postcondition** = musí být splněn **po** vykonání operace

```
context <classifier>::<operation>(<parameters>)
post [<constraint_name>]: <Boolean_OCL_expression>
```

OCL: Precondition / Postcondition (příklady)

```
context Meeting::shift(d : Integer)
pre: self.isConfirmed = false
```

```
context Meeting::shift(d : Integer)
pre positiveDelta: d > 0
```

```
context Meeting::shift(d : Integer)
pre: self.isConfirmed = false and d > 0
```

```
context Meeting::duration(): Integer
post: result = self.end - self.start
```

- `result` označuje výsledek operace

OCL: Precondition / Postcondition (příklady)

```
context Meeting::confirm()
post: self.isConfirmed = true
```

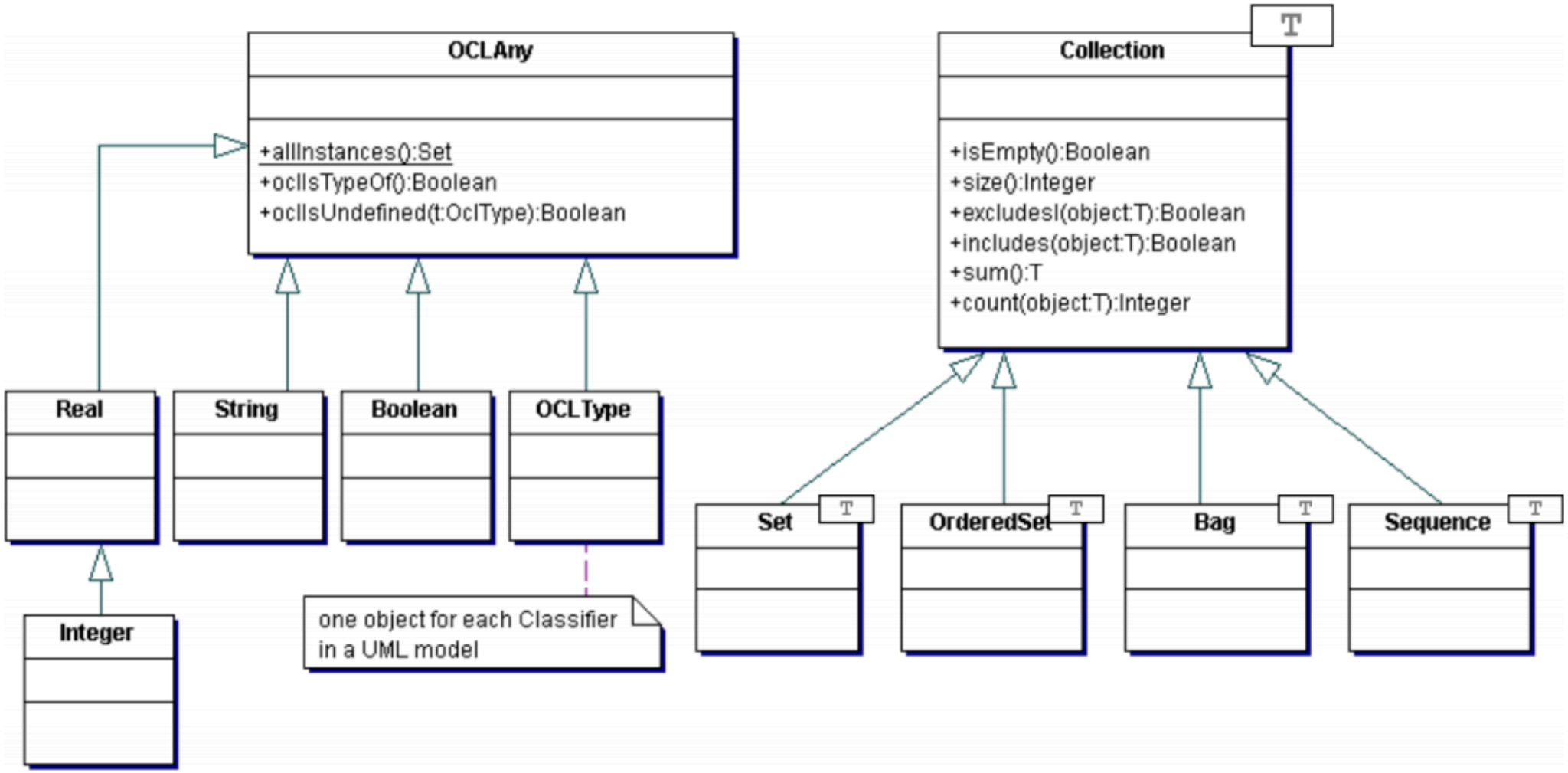
```
context Meeting::shift(d : Integer)
post: start = start@pre + d and end = end@pre + d
```

- `start@pre` = hodnota `start` před vykonáním operace.
- Používá se pouze u postconditions.

OCL: Výrazy

- Boolean výrazy: `and`, `or`, `not`, `xor`, `implies`, `if`-`then`-`else`
- Standardní knihovna primitivních typů a operací.

OCL: Typy


OCLAny
+allInstances():Set
+oclIsTypeOf():Boolean
+oclIsUndefined(t:OclType):Boolean
Collection
T
+isEmpty():Boolean
+size():Integer
+excludesI(object:T):Boolean
+includes(object:T):Boolean
+sum():T
+count(object:T):Integer
Real
String
Boolean
OCLType
Integer
one object for each Classifier
in a UML model
Set
OrderedSet
Bag
Sequence

OCL: Třídní (modelové) typy - OCLType

- třída: např. `Meeting`
- (instanční) atributy: např. `start` (resp. `self.start`)
- operace: např. `duration()` (resp. `self->duration()`)
- třídní atributy: např. `Date::today`
- třídní operace: např. `OCLAny::allInstances()`
- navigace přes asociace: např. `Meeting.participants`
- výčtový typ (enumeration): např. `Gender`, `Gender::male`

OCL: Navigace přes asociace




```
context Meeting
inv: self.moderator.gender = Gender::female
```

- asociace s multiplicitou 1 → objekt
- asociace s multiplicitou > 1 → kolekce

```
context Meeting
inv: self.participants->size() >= 2
```

OCL: Operace nad kolekcemi

- Jsou analogické těm, které znáte z knihoven OO jazyků.
- Literál kolekce: `Set{4, 6, 5}`, `Sequence{'first', 'second', 'third'}`
- přidání, odebrání: `including(object)`, `excluding(object)`
- binární operace nad množinami: `union`, `intersection`, `minus`, `symmetricDifference`
- transformace: `asBag()`, `asSet()`, `asOrderedSet()`, `asSequence()`,
- `flatten(): Set{Bag{1,2,2},Bag{2}} → Set{1,2}`
- operace nad `OrderedSet`, `Sequence`: `first()`, `last()`, `indexOf()`
- iterátory: `select(expr)`, `reject(expr)`, `collect(expr)`, `any(expr)`, `exists(expr)`, `one(expr)`, `forAll(expr)`, `isUnique(expr)`, `sortedBy(expr)`
- Pozn.: Jedná se všechno o operace, syntaxe je tedy např. `self.mySet->size()`
- Detaily a další operace viz reference (Eclipse).

OCL: Příklady operací nad kolekcemi

`including ( object : T ) : Collection(T)`

Vrátí kolekci obsahující všechny původní elementy plus nový objekt.

```
Sequence{'a', 'b'}->including('c') -- Sequence{'a', 'b', 'c'}

Bag{'a', 'b'}->including('c') -- Bag{'a', 'c', 'b'}

Set{'a', 'b'}->including('c') -- Set{'a', 'c', 'b'}
```

OCL: Příklady operací nad kolekcemi

`includes ( object : T ) : Boolean`

Vrátí `true` pokud je objekt obsažen, jinak `false`.

```
Sequence{2.3}->includes(2.1) -- false

Sequence{2.0}->includes(2) -- true
```

OCL: Příklady operací nad kolekcemi

`forAll ( expr : OclExpression ) : Boolean`

Vrátí `true` pokud všechny elementy splní podmínku, jinak `false`.

```
Sequence{2.3, 5.2}->forAll(self > 3) -- false

Sequence{2.3, 5.2}->forAll(self > 1.2) -- true
```

Pozn.: `self` nyní odkazuje na jednotlivé iterované objekty. Pokud v pravidle potřebujeme `self` jako odkaz na kontext, použijeme variantu s proměnnou iterátoru na následujícím slide.

OCL: Příklady operací nad kolekcemi

`select ( expr : OclExpression ) : Collection(T)`

Vrátí kolekci se všemi elementy, které splní podmínku.

```
Sequence{1, 2, 3}->select(i : Integer | i > 1) -- Sequence{2, 3}

Bag{1, 2, 3}->select(i : Integer | i > 1 ) -- Bag{3, 2}

OrderedSet{1, 2, 3}->select(i : Integer | i > 1 ) -- OrderedSet{2, 3}

Set{1, 2, 3}->select(i : Integer | i > 1 ) -- Set{3, 2}
```

OCL: Příklady operací nad kolekcemi

```
collect ( expr : OclExpression ) : Collection(T2)
```

Vrátí kolekci obsahující výsledek aplikace výrazu na všechny elementy

```
Sequence{'first', 'second'}->collect(self.toUpper()) -- Sequence{'FIRST', 'SECOND'}
```

OCL: Příklady na kolekce

- `TeamMeeting` musí být organizován pro celý tým:

```
context TeamMeeting
inv: participants->forAll(team = self.for)

context Meeting
inv: oclIsTypeOf(TeamMeeting) implies participants->forAll(team = self.for)
```

OCL: Příklady na kolekce

- Postconditions (`select()`):

```
context TeamMember::numMeeting(): Integer
post: result = meetings->size()

context TeamMember::numConfMeeting(): Integer
post: result = meetings->select(isConfirmed)->size()
```

OCL: Automatické zplošťování kolekcí

- Kolekce se umí samy zplošťovat během operací (`flatten()`):

```
-- (in context of Meeting)
self.participants.meetings
```

- `self.participants` … `Set(Person)`
- `self.participants.meetings` … `Set(Bag(Meeting))`
- → `Set(Meeting)`

OCL: Odvození

- Pomocí odvození (derivation rule) je možné definovat:

- Odvozený atribut (derived attribute):

```
context Team::size:Integer
derive: members->size()
```

- Odvozenou asociaci (derived association):

```
context Meeting::conflict:Set( Meeting )
derive: select(m | m <> self and self.inConflict(m))
```

OCL: Počáteční hodnota

- Specifikace hodnoty, která musí platit při vytvoření objektu (pouze).

```
context Meeting::isConfirmed:Boolean
init: false

context TeamMember::meetings:Set(Meeting)
init: Set{}
```

OCL: Výraz `let`

- Definice lokální proměnné (neměnná, immutable)

```
context Meeting
inv:
  let noConflict: Boolean =
    participants.meetings->
      forAll(m | m <> self and m.isConfirmed implies not self.inConflict(m))
  in isConfirmed implies noConflict
```

OCL: Výraz `def`

- Přidávání nových atributů a operací do modelu pro opakované použití
- Syntaxe shodná s `let`

```
context Meeting
def: noConflict: Boolean =
  participants.meetings->
    forAll(m | m <> self and m.isConfirmed implies not self.inConflict(m))
```

OCL: Testování prázdných atributů

- prázdný atribut: hodnota `OclVoid`
- `oclIsUndefined(): Boolean`

OCL: Balíčky

```
package MeetingExample

context Meeting::isConfirmed:Boolean
init: false

context TeamMember::meetings:Set(Meeting)
init: Set {}

-- ...

endpackage
```

Nástroje pro práci s OCL

- Řada nástrojů, většina ale není udržovaných a aktualizovaných (viz např. Dresden OCL)
- USE: https://sourceforge.net/projects/useocl/
  - OCL interpret, GUI + příkazový jazyk.
  - Nástroje pro vytváření modelů a jejich analýzu.
  - Neumí kreslení ani import modelů, jen vlastní specifikační jazyk.
  - Nabídka spolupráce: FIT RPP projekt pro export USE formátu z OpenPonk.
- Eclipse:
  - MDT/OCL pro EMF modely.
  - Dresden OCL (TU Dresden): umí kontrolovat XMI formát modelů (Eclipse UML2 Tools, Enterprise Architect, Menthor, OpenPonk, ...).
- Bakalářská práce OCL pro OpenPonk – základ interpretu, příležitost pokračovat.

Časté otázky

„K čemu je OCL, když můžu pravidlo napsat česky nebo rovnou jako test?“

OCL přesně určí, kterých instancí se pravidlo týká a jaká podmínka má platit. Přirozený jazyk vysvětlí záměr a testy ověří vybrané případy; formální pravidlo vyjadřuje obecný požadavek. Tyto způsoby se mohou doplňovat.

Časté otázky

„Když v postcondition napíšu `isConfirmed = true`, provede se tím potvrzení?“

Ne: OCL nemění stav. Postcondition říká, co musí platit po provedení operace; změnu musí zajistit její implementace. Je to smluvený výsledek, nikoli příkaz k přiřazení.

Konec přednášky